separate from each other. Organise your dependencies into separate modules to stop any potential conflicts between different dependencies.

Project structure is:

```
project/
|— data/  #raw and processed documents
|— embeddings/        #embedding logic
|— vector_store/      # database setup
|— chains/        # LangChain pipelines
|— app.py         # main application
```

Data processing and embeddings pipeline

The process of preparing and organising data directly influences the performance of any system that uses LLMs. Raw data must be transformed into three distinct formats, which make it accessible for searching and creating structured data and developing semantic understanding.

The first step begins with document ingestion, which involves gathering and cleaning data from multiple data sources including PDFs, web pages and internal files. This preprocessing step eliminates all noise elements from raw data because it extracts only valuable information. After the data is cleaned, it is broken into smaller parts through a process known as chunking. LLMs and retrieval systems perform better when they use smaller segments of data which maintain context instead of processing complete unstructured documents.

The system creates an embedding for every text segment after the chunking process. Machines use numerical representations called embeddings to understand relationships between different pieces of information that capture the semantic meaning of text. The system stores these embeddings in vector databases, which include FAISS, Pinecone, and Weaviate. These systems enable similarity-based retrieval, which allows users to find relevant information even when they search with keywords different than the actual database content.

User queries are transformed into embeddings, which are used to search for matching vectors in the database. The system retrieves the most relevant chunks based on semantic similarity, which are then passed to the LLM as context for generating a response.

The pipeline converts unstructured data into a format that LLMs can utilise. Data processing and embedding techniques must be designed properly, because they determine the system's success rates in data processing.

Building the application with LangChain

After establishing the data pipeline, developers must integrate all components to create a functional application. LangChain functions as the essential component that unifies user input processing and information retrieval along with answer generation into a continuous operational process.

Developers use LangChain to create predefined workflows called 'chains'. The chains handle all the steps needed to process a query, which starts with a user input and continues until the system produces an answer through an LLM after fetching data from the vector database. Developers can construct pipelines more effectively with LangChain because it offers prebuilt components.

Prompt design is critical at this stage. The system uses prompts, which include retrieved context, instead of sending raw queries to the model. The model can respond to questions by using only the documents the system has provided. This helps reduce hallucinations and ensures that responses are grounded in actual data.

LangChain introduces retrievers, which make it easier to access vector databases. Developers can use retriever interfaces to obtain relevant data from systems such as FAISS and Pinecone without needing to conduct direct system queries. Users can switch databases or change retrieval methods (since the system allows them to do so) without needing to change fundamental system functions.

Another powerful feature is memory, which enables applications to keep track of ongoing interactions through multiple conversations. Understanding previous messages enables better response quality and coherent answers to users. The LangChain platform provides advanced users with agent capabilities, which enable real-time decision-making with respect to tool selection and action based on incoming queries.

This system enables dynamic data retrieval and domain-specific reasoning through its architecture because it combines static knowledge with active data retrieval capabilities. It uses RAG to produce fluent and factual responses, achieving high reliability through hallucination reduction.

The combination of LLMs with orchestration frameworks and vector databases helps establish a standard development pattern for contemporary AI systems. LLMs provide reasoning abilities, vector databases function as memory systems, and LangChain frameworks link these components to create effective real-world solutions. END

By: D. Banerjee and Sourav K.

D. Banerjee is an IT professional with 15+ years of experience in Tier-1 IT companies, specialising in delivering scalable enterprise solutions and leading complex technology initiatives.

Sourav K. is an IT professional with around 5 years of experience in Tier-1 IT companies, skilled in modern technologies and focused on building efficient, data-driven solutions.